PUBLIC

Code Assessment

of the Liquidity Delivery Smart Contracts

January 27, 2026

Produced for

by CHAINSECURITY

Contents

1 Executive Summary 3
2 Assessment Overview 5
3 Limitations and use of report 12
4 Terminology 13
5 Open Findings 14
6 Resolved Findings 16
7 Informational 24
8 Notes 27

1 Executive Summary

Dear M0 Team,

Thank you for trusting us to help you with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Liquidity Delivery according to Scope to support you in forming an opinion on their security risks.

M0 implements the M-Portal-v2 and Order Book to act as a cross-chain liquidity delivery protocol. The M-Portal supports bridging assets and messages across different bridge adapters. The Order Book is used to facilitate intent-based trading and bridging. Importantly, the Order Book supports partial fills. This allows bridging of large amounts in multiple rounds.

The most critical subjects covered in our audit are asset solvency, functional correctness, and correct integration with external bridges. Asset solvency is now high, after addressing Refunded Local Orders Can Be Filled to Steal Funds and Wormhole Messages Can Be Replayed Across Chains to Mint Infinite Tokens. Functional correctness is now high, after addressing FillOrder always reverts when reporting cross-chain fills and Leftover Mapping in setBridgeChainId. The correctness of external bridge integrations is now high, after addressing Wormhole Messages Can Be Replayed Across Chains to Mint Infinite Tokens.

The general subjects covered are access control, front-running, and signature handling. Security regarding access control is high. Security regarding front-running and signature handling was improved by addressing the griefing vector detailed in Griefing Vector in cancelOrderFor.

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical-Severity Findings	2
• Code Corrected	2
High-Severity Findings	0
Medium-Severity Findings	2
• Code Corrected	1
• Specification Changed	1
Low-Severity Findings	8
• Code Corrected	3
• Specification Changed	3
• Risk Accepted	2

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

M Portal V2

The following files are in scope:

```
evm/src/
    bridgeAdapters/
        hyperlane/HyperlaneBridgeAdapter.sol
        wormhole/WormholeBridgeAdapter.sol
        BridgeAdapter.sol

    interfaces/
        IBridgeAdapter.sol
        IHubPortal.sol
        IMerkleTreeBuilderLike.sol
        IMTokenLike.sol
        IOrderBookLike.sol
        IPortal.sol
        IRegistrarLike.sol
        ISpokeMTokenLike.sol
        ISpokePortal.sol
        ISwapFacilityLike.sol

    libraries/
        BytesParser.sol
        PayloadEncoder.sol
        TypeConverter.sol

    utils/ReentrancyLock.sol

    HubPortal.sol
    Portal.sol
    SpokePortal.sol
```

For the solidity smart contracts, the compiler version `0.8.30` was chosen.

Order Book

The following files are in scope:

```
evm/src/
    OrderBook.sol

    interfaces/IOrderBook.sol
```

For the solidity smart contracts, the compiler version `0.8.33` was chosen.

The assessment was performed on the source code files inside the Liquidity Delivery repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

M Portal V2

V	Date	Commit Hash	Note
1	08 Dec 2025	b9799699cab6986e09beee4833423510d3128c1d	Initial Version
2	11 Jan 2026	9ab37d82b4d2450de690a9006302cdfd76a3e8ea	Fixes
3	26 Jan 2026	ad2d43bbd72b8555a0c6b674f32367ef7cca0617	Final Version

Liquidity Delivery

V	Date	Commit Hash	Note
1	05 Dec 2025	15a972bc51a559824b699461c9cdd706badd8491	Initial Version
2	11 Jan 2026	53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566	Fixes
3	22 Jan 2026	38fc29728091d8df28a6f4d3138e5982900dca77	Final Version

2.1.1 Excluded from scope

Any file not explicitly mentioned in the scope is assumed to be out of scope.

Additionally, the following parts are out of scope and assumed to work correctly:

- The $M protocol.
- The Hyperlane bridge.
- The Wormhole bridge.

2.2 System Overview

This system overview describes the initially received version (Version 1) of the contracts as defined in the Assessment Overview.

M0 provides a generic bridging interface for $M and $M extensions together with a cross-chain order book.

For details about the M0 ecosystem and the functioning of the $M stablecoin and its extensions, see our report on the M0 protocol and our $M extensions report.

The bridging interface contains two parts: The HubPortal and the SpokePortal. $M Tokens are locked/unlocked on the hub chain and minted/burned on the spokes. Tokens and messages can be sent between any chains, including spoke-to-spoke. (In Version 2 spokes are isolated by default and must be explicitly enabled for spoke-to-spoke transfers). The system abstracts away the actual bridge implementation by using pluggable bridge adapters, responsible for packing the messages in the correct format. Currently, the supported bridge services are Hyperlane and Wormhole.

The Portal is leveraged by an intent-based order book, which allows same-chain and cross-chain orders. The order book enables any user to exchange any token for another one by getting their order fully or partially filled by a solver.

2.2.1 Portal

The Portal is the base contract that implements logic shared by the Hub and Spoke Portals. It acts as the entry point to send cross-chain messages. Messages can arrive in any order.

The main functions exposed by the Portal are:

- `sendToken()`: Transfers $M Token or $M Extensions to a destination chain, using either the default or a specified bridge adapter. If $M extensions are sent, they are first swapped to $M token using the `SwapFacility`. On the hub, the $M tokens sent are locked in the contract, while on the spoke they are burned. Afterwards, a message is sent to the destination chain, where $M is unlocked (on the Hub) or minted (on the Spoke).
- `sendFillReport()`: Sends a fill report to a destination chain. This function is used by the OrderBook contract.
- `receiveMessage()`: Receives a message from a bridge adapter. If an $M extension transfer is received, $M is wrapped into $M extension before sending to the recipient. In case this fails, $M is sent instead.
- `quote()`: Returns the fee for delivering a cross-chain message, using either the default or a specified bridge adapter. This function is not supported for the WormholeBridgeAdapter, as Wormhole quotes happen off-chain.
- `pause()` and `unpause()`: Allow the pauser to pause and unpause the contract, preventing users from initiating bridging operations (in Version 2, sending and receiving can be paused independently).

2.2.2 HubPortal

The HubPortal is deployed on the hub chain where the $M token is minted, and the original registrar is deployed. The contract locks $M tokens when sending them to a spoke chain and unlocks them when receiving from a spoke chain.

The main functions exposed by the HubPortal are:

- `sendMTokenIndex()`: Sends the $M token index to a destination spoke chain.
- `sendRegistrarKey()`: Sends a registrar key to a destination spoke chain. The value of the key is fetched from the registrar on the hub chain.
- `sendRegistrarListStatus()`: Sends the registrar list status for a given list and an account to a destination chain. The status is fetched from the registrar on the hub chain.
- `enableEarning()`: Enables earning for the Hub Portal if allowed by the Two Token Governance (TTG).
- `disableEarning()`: Disables earning for the Hub Portal if disallowed by the TTG.

2.2.3 SpokePortal

The SpokePortal is deployed on spoke chains and mirrors the hub state locally. When sending/receiving $M, the spoke Portal burns/mints the tokens.

The main functions of the SpokePortal are:

- `_updateMTokenIndex()`: Is called by `receiveMessage()`. It updates the $M token index on the spoke chain.
- `_setRegistrarKey()`: Is called by `receiveMessage()` to set a registrar key.
- `_updateListStatus()`: Is called by `receiveMessage()` to update a registrar list status.

The spoke chain's $M token index is also updated every time a `sendToken` message is received from the hub chain (in Version 2, all send functions were updated to also include the index, including spoke-to-spoke transfers).

2.2.4 BridgeAdapter

`BridgeAdapter` is an abstract contract for cross-chain messaging backends. It defines a common interface for sending and receiving messages.

This contract can be inherited by bridge-specific adapters, which must implement the `sendMessage()` function with bridge-specific logic.

At the moment, two bridge adapters are implemented:

- `HyperlaneBridgeAdapter`: sends and receives messages through the Hyperlane mailbox. Relies on Hyperlane's default Interchain Security Module (ISM) for message verification.
- `WormholeBridgeAdapter`: sends and receives messages through the Wormhole core bridge. Relies on the Wormhole core bridge for message verification.

2.2.5 OrderBook

The OrderBook allows users to place same-chain and cross-chain orders. It supports partial fills, which allows bridging large amounts in multiple rounds.

Users can place orders specifying an amount of `tokenIn` to receive in exchange for an amount of `tokenOut`. Opening an order generates an event, which can be observed off-chain by an order solver (filler). The solver can then fill the order by filling a certain amount of `tokenOut` to receive a proportional amount of `tokenIn`.

Same-chain Orders In case the order origin chain is the same as the order destination chain, no cross-chain messages are exchanged. The solver can fill the order by sending an amount of `tokenOut` to the order sender and gets reimbursed with `tokenIn` immediately.

Cross-chain Orders In case the order's origin chain is different from its destination chain, orders are filled asynchronously. The solver fills the order up-front by paying `tokenOut` on the destination chain. This triggers a cross-chain message sent through the Portal to the origin chain. Once the message is received, the solver gets reimbursed with `tokenIn` on the origin chain. There is a `finalityBuffer` configured per chain. It specifies how long an order must wait to be refunded after being canceled or expiring (in Version 2, cancellation was moved to the destination chain and no longer requires a finality buffer). It is expected to be configured in such a way that cross-chain messages have enough time to be delivered before the buffer expires. Solvers must track events emitted by the OrderBook. See Solvers must track events carefully.

The OrderBook exposes the following functions:

- `openOrder()`: Opens an order with the given parameters. The caller is set as the sender of the order and pays the `amountIn` of tokens to open the order. The order data is stored in the contract, and an event is emitted for solvers.
- `openOrderWithPermit()`: Approves `amountIn` of input tokens using a permit signature and opens an order.
- `openOrderFor()`: Opens an order on behalf of the order sender. The caller must provide GaslessOrderParams signed by the order sender.
- `openOrderForWithPermit()`: Approves `amountIn` of input tokens using a permit signature and opens an order on behalf of the order sender. The caller must provide GaslessOrderParams signed by the order sender.
- `fillOrder()`: Fills an order. If the order is a same-chain order, the filler gets immediately refunded based on the amount filled. If the order is a cross-chain order, the function calls `Portal.sendFillReport()`, sending a cross-chain message.
- `reportFill()`: Receives a fill report from the destination chain of the given order. The function updates the order status and filled amounts in storage and refunds the filler of the order on the source chain.

- `requestCancelOrder()`: Requests cancellation of an order. After the `finalityBuffer` has passed since the cancellation request, the order sender can call `claimRefund()` to get refunded. During the buffer period, the order can still be filled by solvers. (In Version 2, cancellation was moved to the destination chain and the finality buffer was removed.)
- `requestCancelOrderFor()`: Cancels an order on behalf of the order sender. The caller must provide an orderId to cancel, signed by the order sender. (In Version 2, this was replaced by `cancelOrderFor` which uses the order recipient's signature instead.)
- `claimRefund()`: Can be called by anyone to trigger a refund of an expired or canceled order to the order sender once its `finalityBuffer` has passed. (In Version 2, this function was removed. Refunds are now handled in `reportFill()`.)

The permit functions only support tokens that implement EIP-2612 permits.

2.2.6 Changes in Version 2

The following changes were introduced in Version 2 of the contracts:

Portal:

Spoke Isolation

- Spoke chains are now isolated by default, meaning that they can only transfer tokens to the Hub, not to other spokes. Once a spoke's isolation is disabled, it cannot be re-enabled.
- The HubPortal now maintains per-spoke tracking of bridged principal for isolated spokes, preventing them from bridging back more than was transferred there.
- SpokePortals now also track their own isolated status, the status of other spokes, and store the Hub's address for reference.
- Spokes now enforce that they cannot send to isolated spokes.

Extended Index Synchronization

- All send functions from Hub to Spoke and from Spoke to Spoke now include the $M token index in the payload.
- The receiver Spoke now updates the index if greater than the current.

Solana Compatibility

- The system now supports sending the merkle root of the registrar list of Solana earners to the SpokePortal on Solana.

Granular Pausability

- The pauser can now selectively pause sending and receiving operations independently.

Fee-on-Transfer Token Rejection

- The Portal now explicitly rejects fee-on-transfer tokens. The `_transferAndUnwrap()` function reverts if the actual transferred amount is less than specified.
- Additional validation has been added to ensure that if the amount of $M received differs from the specified amount beyond the rounding threshold, the transaction reverts.

OrderBook

Pausability

- The order book can now be paused. If paused, the `_openOrder`, `_fillOrder`, and `_cancel` functions are not callable. Reports that have already been sent are still accepted by receiving chains.

Order Cancellation

- Order cancellations are now submitted on the destination chain. This approach eliminates the need for a finality buffer.
- Gasless order cancellation is now supported through the `cancelOrderFor` functions, allowing anyone to cancel an order by providing a signature of the order recipient.

2.2.7 Changes in Version 3

The following changes were introduced in Version 3 of the contracts:

Portal:

- In addition to not sending to isolated spokes, spokes now enforce that they do not receive tokens from other spokes if the sender is isolated or if they are isolated themselves.
- The HubPortal now implements a migration mode, allowing operators to manually set the bridged principal for each isolated spoke chain. This allows isolated spokes to bridge tokens back to the hub after migration. After setting all the necessary principal amounts, an operator can call `completeMigration` to permanently disable the migration mode.

2.3 Trust Model

2.3.1 Portal

- Admin: fully trusted. They can upgrade the implementation of the Portal. In the worst case, a corrupted admin could upgrade the Portal implementation, drain all locked tokens, and mint unlimited $M on spoke chains. They are also trusted to set the correct gas limits for each chain.
- Operators: fully trusted. They can set supported bridging adapters, default bridge adapters, peers, supported bridging paths, payload gas limits, and disable spoke isolation. In the worst case, a corrupted operator could set a malicious bridge adapter and drain the HubPortal or mint unlimited $M on spoke chains.
- Pauser: partially trusted. They can pause the Portal. In the worst case, it could pause the contract, preventing users from sending cross-chain messages until the admin intervenes. However, even with a paused system, messages could still be received.

Assumptions

- The SpokePortal is assumed to have either the Governor or the Emergency Governor role in the TTG registrar in order to set the $M token index and mint tokens.
- Hyperlane is fully trusted and assumed to work as specified. Hyperlane can change its default ISM and its validator set at any time.
- Wormhole is fully trusted and assumed to work as specified. Wormhole can change its guardian set at any time.

2.3.2 OrderBook

- Proxy admin: fully trusted. They can upgrade the implementation of the OrderBook. In the worst case, a corrupted proxy admin could upgrade the OrderBook implementation to a malicious one and drain all funds held by the contract.
- Admin: partially trusted. In the worst case, they could cause a temporary denial of sevice by disabling a destination chain that should be enabled.
- Solvers: untrusted.

2.3.3 Tokens

- For the Portal, only $M and $M extensions are supported. Reentrant tokens (e.g., ERC777) and tokens with transfer fees are not supported. Tokens that can have negative rebases are not supported.
- For the OrderBook, any ERC20 token can be used (there is no limitation in the contract). However, the contract is only designed to handle standard ERC20 tokens without special behaviors. In particular, rebasing tokens (including $M) and tokens with transfer fees are not supported. Fillers are responsible for ensuring they do not fill orders that use malicious tokens. The contract is assumed not to be blacklisted by any token.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the Resolved Findings section. The findings are split into these different categories:

- Security: Related to vulnerabilities that could be exploited by malicious actors
- Design: Architectural shortcomings and design inefficiencies
- Correctness: Mismatches between specification and implementation

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	2

- Discrepancy Between Tracked and Actual Principal Risk Accepted
- Discrepancy of Index When Earning Is Disabled Risk Accepted

5.1 Discrepancy Between Tracked and Actual Principal

Design Low Version 1 Risk Accepted

CS-M0-LIQ-018

In the `HubPortal`, the principal amount that is tracked for each Spoke might diverge from the actual transferred principal amount of `MToken`.

Specifically, the tracked principal for a given Spoke chain is decremented as follows:

```
uint248 totalBridgedPrincipal = spokeConfig.bridgedPrincipal;
uint248 principalAmount = IndexingMath.getPrincipalAmountRoundedDown(uint240(amount), _currentIndex());

// Prevents unlocking more than was bridged to the Spoke
if (principalAmount > totalBridgedPrincipal) revert InsufficientBridgedBalance();

unchecked {
    spokeConfig.bridgedPrincipal = totalBridgedPrincipal - principalAmount;
}
```

Where `principalAmount`, subtracted from `bridgedPrincipal`, is given by converting `amount` to principal. Note that the result of this operation is rounded down.

As a consequence, it is possible to bridge back some non-zero amount and not decrease the internally tracked principal (`bridgedPrincipal`). In fact, the bridge can theoretically be drained if an adversary can mint arbitrary unbacked `MToken` on a spoke chain ("Spoke is compromised"). However, this attack would not be profitable to an attacker, as the bridge will have to be drained by billions of transactions of `index / EXP_SCALED_ONE` wei each, and the cost of each transaction would be greater than the profit.

Risk Accepted: M0 has accepted the risk stating:

```
[...] we consider the described attack highly unlikely, as an attacker
 would spend more on gas and bridging fees than they could ever extract.
```

5.2 Discrepancy of Index When Earning Is Disabled

Correctness **Low** **Version 1** **Risk Accepted**

CS-M0-LIQ-008

If the governance removes the `HubPortal` from the earners list, anyone can trigger a call to stop the portal from earning more yield in `MToken` in two ways:

1. Call `disableEarning()` in the `HubPortal` wrapper contracts.
2. Call `stopEarning()` in `M` token contract.

The first option is the correct way to stop the wrapper from earning more yield and ensure that the reported index is correct. However, since anyone can call `stopEarning()` in the `M` token contract, the second option is also possible. In this case, the index bridged to the Spoke chains will be too big and will still grow until `disableEarning()` is executed. Theoretically, any delay might create solvency issues for the `HubPortal` as the yield accrued might become smaller than the yield accrued on the Spoke chains. Hence, the last users would not be able to bridge back their tokens, as some of the tokens on the Spoke chains would become unbacked.

Risk accepted:

M0 accepted this risk and stated:

```
Removing HubPortal from the Earner list is a highly unlikely event. If it
does occur, the M0 engineering team will monitor the Registrar list update and execute
the disableEarning() call accordingly.
```

6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the Open Findings section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	2

- Refunded Local Orders Can Be Filled to Steal Funds Code Corrected
- Wormhole Messages Can Be Replayed Across Chains to Mint Infinite Tokens Code Corrected

High-Severity Findings	0

Medium-Severity Findings	2

- Fee on Transfer Extension Tokens Can Cause Reverts Specification Changed
- FillOrder Always Reverts When Reporting Cross-Chain Fills Code Corrected

Low-Severity Findings	6

- Dangerous Comment in fillOrder Specification Changed
- Funds Cannot Be Bridged Back to Hub After Upgrade Specification Changed
- Griefing Vector in cancelOrderFor Code Corrected
- Leftover Mapping in setBridgeChainId Code Corrected
- No Isolation Validation on Receiving Side Specification Changed
- Version Is Not Checked in openOrderFor Code Corrected

Informational Findings	4

- Outdated Code Comments Specification Changed
- Inconsistent Admin Naming Code Corrected
- Missing Zero Check for Origin Recipient Code Corrected
- Missing _disableInitializers in OrderBook Constructor Code Corrected

6.1 Refunded Local Orders Can Be Filled to Steal Funds

Correctness Critical Version 1 Code Corrected

CS-M0-LIQ-001

In OrderBook, when a user calls `claimRefund()`, the order's status is updated to `Completed`, and funds are returned to the user. The `filledAmounts` mapping is not updated to reflect this payout. This is ok, as `Completed` orders should not be fillable.

However, the `_fillOrder()` function does not check the order status when filling a local (same chain) order. As a result, a `Completed` order can still be filled for the amount that was refunded (`orderData.amountIn - filledAmounts.amountInReleased`).

Consider the following exploit scenario, with an attacker A:

1. A opens a local order with 1000 tokenIn and 1000 tokenOut. A transfers 1000 tokenIn to the OrderBook contract.
2. A cancels and refunds the order, receiving the 1000 tokenIn back. Status becomes `Completed`.
3. A calls `fillOrder` on the same order, acting as the solver.
4. Since `filledAmounts.amountOutFilled` is still 0, the contract calculates the full principal as available to be filled.
5. The contract transfers 1000 tokenIn (stolen from other users' deposits) to A. A transfers 1000 tokenOut to themselves (as they fill their own order).
6. A has stolen 1000 tokenIn from the OrderBook contract.

Repeating this attack allows the attacker to steal all tokens from the OrderBook contract.

This issue does not affect cross-chain orders, as the `reportFill` function reverts if the order status is `Completed`.

Code corrected:

The code now updates the filled amounts on both order filling and cancellation. Also, canceled orders can no longer be filled.

Cancelling an order updates the `filledAmounts.amountInRefunded` and sets the order as `Cancelled`.

A same-chain order in the `Cancelled` status cannot be filled, while for cross-chain orders, the function `_reportFill` validates that the total amount released plus the total amount reimbursed (due to cancellation) does not exceed `order.amountIn`.

6.2 Wormhole Messages Can Be Replayed Across Chains to Mint Infinite Tokens

Correctness | Critical | Version 1 | Code Corrected

CS-M0-LIQ-002

In WormholeBridgeAdapter, the `executeVAAv1` function is permissionless. It ensures that the passed VAA is valid by calling `ICoreBridge.parseAndVerifyVM`.

The VM struct that is verified contains information about the emitter chain ID and emitter address, but does not contain any information about the recipient chain ID or recipient address. The default behavior of Wormhole is to "multicast" messages to all chains, so the VAA does not specify a recipient.

In order to restrict execution to only intended chains and recipients, they are expected to be passed as part of the payload and verified in the receiving contract. However, in WormholeBridgeAdapter, this is not implemented.

As a result, a message can be emitted once and then executed on multiple chains. The conditions that must be met for this to be possible are:

1. The WormholeBridgeAdapter contract is deployed on the sending chain and multiple receiving chains.
2. The receiving chains have the same `peer` address configured for the sending chain.
3. The message payload has an effect on multiple chains (e.g., minting $M tokens).

Conditions 1. and 2. are the expected setup. 3. is also fulfilled, as `sendToken` messages can mint $M tokens independently of which chain they are executed on (even in case the destination $M-extension token does not exist).

The following attack scenario illustrates how this vulnerability can be used to duplicate tokens:

1. The attacker calls `sendToken` on the Portal to send 100 tokens from Chain A to Chain B.
2. `executeVAAv1` will be called by a Wormhole relayer on Chain B, minting 100 tokens to the attacker.
3. The attacker calls `executeVAAv1` again on Chain C, using the same VAA. An additional 100 tokens are minted to the attacker on Chain C.
4. The attacker can bridge back the tokens from both chains to receive 200 tokens on chain A. They have duplicated their tokens.

By repeating this process, the attacker can mint unlimited tokens on Spoke chains and drain all $M tokens from the HubPortal.

Code corrected:

M0 has corrected the code by including the destination chain ID and the destination address inside the cross-chain message payload. When a message is delivered to the WormholeBridgeAdapter, the `executeVAAv1` function decodes the payload and checks that the destination chain ID and destination address match the ones specified in the payload by the source chain:

```
(uint32 targetChainId, bytes32 targetBridgeAdapter) = vm.payload.decodeDestinationChainIdAndPeer();
if (targetChainId != IPortal(portal).currentChainId()) revert InvalidTargetChain(targetChainId);
if (targetBridgeAdapter.toAddress() != address(this)) revert InvalidTargetBridgeAdapter(targetBridgeAdapter);
```

6.3 Fee on Transfer Extension Tokens Can Cause Reverts

Correctness **Medium** **Version 1** **Specification Changed**

CS-M0-LIQ-003

The Portal contract supports `$M` extension tokens that implement a fee-on-transfer mechanism.

However, the following code in `_sendToken()` will revert in case there is a fee-on-transfer:

```
IERC20(sourceToken).transferFrom(msg.sender, address(this), amount);

    // If the source token isn't $M token, unwrap it
    if (sourceToken != address(mToken)) {
        IERC20(sourceToken).approve(swapFacility, amount);
        ISwapFacilityLike(swapFacility).swapOutM(sourceToken, amount, address(this));
    }
```

In `transferFrom()`, the contract will receive slightly less than `amount`. Given that the contract did not have an initial balance of the `sourceToken`, the `swapOutM()` call will revert, as the contract's balance of `sourceToken` is less than `amount`.

The contract is not expected to hold any balance of `$M` extension tokens, so reverts are likely.

Specification changed:

As of Version 2, fee-on-transfer tokens are no longer supported by the Portal contract. It now intentionally reverts when an M-extension token with fee-on-transfer is used as a source token.

6.4 FillOrder Always Reverts When Reporting Cross-Chain Fills

Correctness | Medium | Version 1 | Code Corrected

CS-M0-LIQ-004

In OrderBook, the `fillOrder` functions trigger a fill report back to the origin chain if it is a cross-chain order. However, these functions will always revert when attempting to fill a cross-chain order.

The implementation fails to account for the following:

- **Missing Payable Modifier**: Cross-chain messages sent via the Portal require a bridge fee in the native token. However, neither `fillOrder` function is declared as `payable`, preventing them from receiving the necessary fees from solvers.
- **Missing Value Passing**: The internal `_fillOrder` function calls `IMessenger.sendFillReport()` without passing any `msg.value`. Since the bridge requires a fee to process the message, the call will always fail.
- **Interface and Signature Mismatch**: The IMessenger interface used by OrderBook defines `sendFillReport()` with only three arguments and is missing the `refundAddress` parameter. In contrast, the Portal contract's implementation requires four arguments. This mismatch in the function signature will cause the call to revert.

As a result, solvers are unable to fill any cross-chain orders, as they will always revert.

Code corrected:

M0 has corrected the code by:

- adding the `payable` modifier to all external functions which require a bridging fee.
- passing the `msg.value` and using the correct function signature when calling the functions in Portal.

6.5 Dangerous Comment in fillOrder

Security | Low | Version 1 | Specification Changed

CS-M0-LIQ-005

In the OrderBook contract, the `_fillOrder()` function contains the following comment:

```
// Ensure the provided order ID matches the computed order ID from the order data
// This check is not strictly required, but it is a useful sanity check for solvers
// to ensure they have the order data correct
if (orderId_ != getOrderId(orderData_)) revert OrderIdMismatch();
```

According to the comment, this check is not strictly required.

However, removing this check would allow an attacker to provide mismatched order data and order ID. This would lead to critical security vulnerabilities. For example, an attacker could fill an order with an incorrect recipient address, then report the fill with the correct order ID to get reimbursed.

The comment should be updated to reflect that this is a critical security check. The check should not be removed in future versions.

Specification changed:

M0 has updated the comment to clarify that the check is necessary to prevent the caller from specifying malicious order data.

6.6 Funds Cannot Be Bridged Back to Hub After Upgrade

Correctness | Low | Version 1 | Specification Changed

CS-M0-LIQ-020

After migrating from V1 of the Portal to V2, the HubPortal's `bridgedPrincipal` mapping will be zero for all spoke chains. V2 uses this value to enforce chain isolation: when receiving tokens from an isolated spoke, `_decreaseBridgedPrincipal` reverts if the unlock amount exceeds `bridgedPrincipal`. Since V2 has no setter function for `bridgedPrincipal`, it cannot be initialized with V1's bridged amounts during migration or configuration.

This means that, for chains that are set as isolated in V2, users who bridged M tokens to that chain in V1 would be unable to bridge them back to the hub after the V2 upgrade.

Specification changed:

A migration mode has been added. It allows operators to manually set the bridged principal for each isolated spoke through `migrateBridgedPrincipal`. The function is disabled once migration is complete. This solves the issue.

6.7 Griefing Vector in cancelOrderFor

Design | Low | Version 1 | Code Corrected

CS-M0-LIQ-024

In `cancelOrderFor`, the recipient signature does not include the `bridgeAdapter` and `BridgeAdapterArgs`. If an attacker observes this signature in the mempool and calls the function with a different bridge or bridge adapter arguments, the execution on the other chain might fail.

Specifically, if the bridge adapter arguments for the Wormhole adapter are incorrect, the automatic execution on the other chain will fail and the user will have to manually execute the message.

Code corrected:

The `bridgeAdapter` and `BridgeAdapterArgs` are now included in the `_cancelOrderFor` signature. This way, the cancellation request can only be executed if the bridge adapter and its arguments match the ones signed by the recipient.

6.8 Leftover Mapping in `setBridgeChainId`

Design | Low | Version 1 | Code Corrected

CS-M0-LIQ-006

The function `setBridgeChainId()` maps a Portal chainID to a bridge-specific chainID by filling the mappings `internalToBridgeChainId` and `bridgeToInternalChainId`. The IDs are mapped bi-directionally, meaning that if ID A is mapped to B, then B is also mapped to A:

```
setBridgeChainId(A, B):
    A -> B
    B -> A
```

However, in case A is later remapped by calling:

```
setBridgeChainId(A, C):
    A -> C
    C -> A

    (still, B -> A)
```

B will still be mapped to A, creating an incorrect mapping.

As the function cannot be called with a zero address as input, it is impossible to fix the inconsistency by mapping B to zero.

Code corrected:

M0 has corrected the code such that `setBridgeChainId()` clears the old mappings, ensuring a consistent state. Following the example above, when mapping A to C, `setBridgeChainId()` looks up the IDs previously mapped to A and C. Since A was previously mapped to B, the function clears the leftover mapping `B -> A`.

6.9 No Isolation Validation on Receiving Side

Design | Low | Version 1 | Specification Changed

CS-M0-LIQ-021

The protocol enforces spoke isolation before `_burnOrLock` via `_revertIfTokenTransferDisabled`, which checks the `crossSpokeTokenTransferEnabled` mapping. However, there is no equivalent check in `_mintOrUnlock`. The `sourceChainId` parameter is received but not validated.

This is ok, as long as peers are correctly configured: isolated spokes should never be added as BridgeAdapter peers of other spokes. If set as a peer, a compromised isolated spoke could disable its local isolation flag, send tokens to the non-isolated spoke, and bridge them back to the Hub. This would break the isolation and allow a compromised isolated spoke to drain all liquidity from the Hub.

A receiving-side check would provide defense-in-depth against an accidental misconfiguration.

Specification changed:

The `_revertIfTokenTransferDisabled` function is now called in `_mintOrUnlock`. This prevents accepting tokens from isolated spokes. Additionally, it prevents isolated spokes from receiving tokens from any spoke.

6.10 Version Is Not Checked in openOrderFor

Correctness **Low** **Version 1** **Code Corrected**

CS-M0-LIQ-007

In OrderBook, the user signs a `GaslessOrderParams` struct that includes a `version` field. However, in the `_openOrderFor` function, the `version` field is not checked against the current contract version.

The function is expected to revert if the `version` in the signed order does not match the contract's current version.

Code corrected:

M0 has corrected the code by checking `orderParams._version` against the current OrderBook version in `_openOrderFor`.

6.11 Outdated Code Comments

Informational **Version 2** **Specification Changed**

CS-M0-LIQ-013

The following comments in the code are outdated or incorrect:

1. In IPortal, the comment for `msgSender()` states that it returns the address of the caller of the `transfer` function. However, there is no `transfer` function. It returns the address of the caller of a function with the `whenNotLocked` modifier (or zero if there has been no such call).
2. In Portal, the comment in `_sendToken()` states that in the case of the HubPortal, the bridged principal is updated. However, the bridged principal is no longer tracked in this version of the contract.
3. In OrderBook, the comment in `_requestCancelOrder()` states that it validates that the caller is the sender. However, this validation does not occur in this function.

The comments should be updated to accurately reflect the current functionality of the code.

In **Version 2**, the code was updated, making some comments outdated:

4. In `_revertIfTokenTransferDisabled` of the SpokePortal, the comments still refer to the old version of the function, used only when sending messages. The usage of the function on message receival is not mentioned and references to the outdated parameter `destinationChainId` are still present. The same problem is present in the natspec of the virtual function `_revertIfTokenTransferDisabled` of the Portal.

Specification changed:

1. The comment has been updated to "The address of the original caller of `sendToken` function". Technically, it could also be the caller of `sendCancelReport` or `sendFillReport`. However, these functions do not make an external call. The `sendToken`

function can call the SwapFacility, which can use the `msgSender()` value. This is the intended use case for this function.

2. As of Version 2, the bridged principal is now tracked in the HubPortal for isolated Spokes.
3. The `_requestCancelOrder()` function has been removed.
4. The comments about `_revertIfTokenTransferDisabled` have been corrected in Version 3.

6.12 Inconsistent Admin Naming

Informational | Version 1 | Code Corrected

CS-M0-LIQ-010

In HyperlaneBridgeAdapter and WormholeBridgeAdapter, the initializer function takes an `owner` parameter and passes it to the `_initialize` function of the BridgeAdapter. However, the BridgeAdapter contract uses the term `admin` to refer to the same role.

Code corrected:

M0 has corrected the code by renaming the input parameter to `admin` in both HyperlaneBridgeAdapter and WormholeBridgeAdapter.

6.13 Missing Zero Check for Origin Recipient

Informational | Version 1 | Code Corrected

CS-M0-LIQ-023

In the `_fillOrder` function, there is no sanity check that ensures the `fillerParams_.originRecipient` parameter is non-zero. If the solver mistakenly sets this address to zero, their refund will be burned.

Code corrected:

The `_fillOrder` function now reverts if `fillerParams_.originRecipient` is zero.

6.14 Missing _disableInitializers in OrderBook Constructor

Informational | Version 1 | Code Corrected

CS-M0-LIQ-022

Both the Portal and the OrderBook contracts will be deployed behind a proxy. The Portal constructor calls `_disableInitializers` to prevent the constructor from being called in the implementation. However, the OrderBook constructor does not call `_disableInitializers`.

Code corrected:

M0 has resolved the issue by adding `_disableInitializers` to the OrderBook constructor.

7 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

7.1 64-Byte EIP-1271 Signatures Not Supported

Informational **Version 1**

CS-M0-LIQ-009

In OrderBook, multiple signature types are supported for order authorization, including EIP-2098 and EIP-1271 signatures.

If a signature is exactly 64 bytes long, it is treated as an EIP-2098 signature. However, EIP-1271 signatures can have an arbitrary length and could also be 64 bytes long. In case a user provides a valid EIP-1271 signature that is exactly 64 bytes, it will be incorrectly interpreted as an EIP-2098 signature, leading to a failed authorization.

While this edge case is theoretically possible, it is not common for EIP-1271 signatures to be exactly 64 bytes. Other signature verification libraries, such as the one used in Uniswap's Permit2, also do not account for this edge case. As a result, it is unlikely anyone would use a 64-byte EIP-1271 signature in practice.

7.2 No Revert for Unsupported Payload Types

Informational **Version 1**

CS-M0-LIQ-019

The `_receiveCustomPayload` function in `SpokePortal` handles `Index`, `RegistrarKey`, and `RegistrarList` payloads, but does not revert for unrecognized types. If an unsupported payload (e.g., `EarnerMerkleRoot`, which is SVM-only) reaches the function, it is silently ignored.

7.3 SpokePortals Must Have Some Token Balance

Informational **Version 1**

CS-M0-LIQ-014

In Portal, the `_sendToken` function intentionally subsidizes rounding errors when transferring `$M` tokens across chains.

The code contains the following comment:

```
[...] the deficit caused by rounding errors will
be covered from the yield earned by HubPortal.
```

For the HubPortal, this is acceptable, as it is expected to earn excess yield over time. However, SpokePortals do not earn yield and are not expected to hold any `$M` balance (they burn any `$M` received when bridging).

The SpokePortal will attempt to burn exactly the `$M` tokens specified by the user. In case it does not have any prior balance, the burn operation can revert due to insufficient balance caused by rounding.

This issue can be mitigated by sending a small amount of `$M` tokens to each SpokePortal to cover rounding losses.

7.4 Transient Storage Required

Informational **Version 1**

CS-M0-LIQ-015

The Portal uses a transient storage reentrancy lock. As a result, the contract can only be deployed on networks that support transient storage (EIP-1153).

In case the contract should be deployed to a chain that does not support it yet, the reentrancy lock would need to be replaced with one that uses normal storage.

7.5 Not All Yield Will Be Claimed

Informational **Version 1** **Acknowledged**

CS-M0-LIQ-017

If there are any $M tokens on Spoke chains that are not earning, the `HubPortal` (assumed to be earning) will be accruing more yield than the amount of yield that can be claimed by holders on Spoke chains. As a consequence, the `MToken` balance of the `HubPortal` will be greater than the amount bridged on the Spoke chain. The difference between those amounts will be locked in the `HubPortal`.

As a note, even if all the holders on Spoke chains are earners, because of time delays induced by the bridge, the index on the Hub chain will always be strictly bigger than the indices on the Spoke chains as long as the `HubPortal` is earning. Additionally, during the bridging process, no yield is earned on sent tokens.

Acknowledged:

M0 has stated:

```
We expect that most of the bridged M tokens will be in earning contracts,
since all governance-approved M extensions are earners. We're aware that
the excess yield locked in HubPortal isn't recoverable, and we're comfortable
with this as we don't anticipate it being a significant amount.
```

7.6 Order Structs Are Filled Out of Order

Informational **Version 1** **Acknowledged**

CS-M0-LIQ-012

The following functions use explicit declarations to initialize Solidity structs in an order that is different from the order declared in the Interface:

- In `_openOrder` when initializing `OrderData` and `Order`.
- In `_openOrderFor` when initializing `OrderParams`.
- In `_fillOrder` when initializing `FillReport`.

While the current code is correct, using the same order as the Interface could increase code readability and prevent mistakes in future versions.

Acknowledged:

M0 is aware of the inconsistency and has decided to leave the code unchanged.

7.7 Wormhole Messages Can Be Intentionally Delayed

Informational | Version 1 | Risk Accepted

CS-M0-LIQ-016

For some messages, such as `sendRegistrarListStatus` and `sendRegistrarKey` in HubPortal, it is a known issue that messages can coincidentally arrive out of order. An older message can be delivered after a newer message. As there is no timestamp included in the message, the state is updated and overwrites the newer state. This is accepted behavior.

However, if the wormhole bridge is used, an attacker can **intentionally** delay messages by not paying for their execution on the destination chain. For example, they could submit an invalid `signedQuote` in the message data. This will cause the message to be signed by wormhole guardians, but it will not be automatically executed on the destination chain. Someone needs to notice the pending message and execute it manually, paying for gas.

Consider the following scenario:

1. An attacker calls `sendRegistrarListStatus` with the `earner` list and the address of a contract that is currently not an earner, but will become one in the future. The attacker includes an invalid signedQuote in the message data.
2. The message is signed by wormhole guardians, but not executed on the destination.
3. Weeks later, the contract has become an earner.
4. The attacker executes the old message, reverting the `earner` list to not include the contract.

This will temporarily revert the `earner` list, potentially causing a disruption. It can be fixed by sending a new message, but this requires someone to notice the issue and pay for gas to fix it. Additionally, the attacker can create any number of such messages and keep them pending indefinitely. In order to solve the issue, all of the messages need to be executed before setting the correct state.

Risk accepted:

M0 is aware of this behavior and accepts it, stating:

```
We accept this limitation since RegistrarKey and RegistrarListStatus messages aren't frequent
and anybody can propagate the correct Key or List Status.
```

8 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

8.1 Funds Could Be Lost Due to Hard Forks

Note Version 1

Both the Portal and the OrderBook rely on cross-chain messaging to function correctly. Generally, messaging protocols wait for a source chain's "finality" before executing the message on the other chain (some even execute earlier). If a hard fork occurs on either the source or destination chain, messages may be executed on only one side of the fork. This can lead to loss of funds.

Consider the following example:

1. A user creates a cross-chain order from Chain A to Chain B.
2. The order is filled on Chain B, and a message is sent to Chain A to release funds to the solver.
3. Chain B "finalizes" the Block in which the fill occurred, so the bridged message can be executed on Chain A.
4. A hard fork occurs on Chain B, which removes the transaction that filled the order from the canonical chain.
5. The message to release funds to the solver has already been executed on Chain A, but the fill on Chain B got reverted. The solver has stolen money from the user.

While waiting for "finality" before executing cross-chain messages protects from temporary chain reorganizations, it does not protect against hard forks caused by node upgrades or bugs. Finality simply means that a node will not switch from the current canonical chain to a different one automatically. However, the canonical chain can still change if a majority of nodes decide to switch to a different chain. For example, by upgrading their software.

While this is an unlikely scenario, it is important to be aware of the risk.

8.2 Incentive System Must Be Permissioned

Note Version 1

In the OrderBook contract, solvers are not expected to make a profit by filling orders. Instead, it is planned that solvers will be compensated through a currently undefined external incentive system. When designing this incentive system, it is crucial to ensure that there is a permissioning mechanism for solvers to receive rewards. If the incentive system is open to all (at no cost), solvers could create orders and fill themselves to earn rewards. This would dilute the rewards for honest solvers that service real users.

As anyone can create an unlimited number of Blockchain addresses, it is impossible to tell whether a solver is filling their own order or someone else's.

8.3 Solvers Must Track Events Carefully

Note Version 1

The OrderBook contract emits the `CancelRequested` event when orders are canceled. Solvers must monitor contract events to maintain an accurate view of the order book state. Cross-chain orders that have been canceled or that do not exist can still be filled by solvers. Solvers must avoid filling such orders, or they risk losing funds.

The following events are critical for solvers to track:

- `OrderOpened`
- `OrderCancelled`
- `OrderFilled`

8.4 Solvers Should Validate Cross-Chain Message Execution

Note **Version 1**

In the OrderBook, solvers fill orders by calling the `fillOrder` function. For cross-chain orders, this sends a message to the origin chain. Solvers must ensure that the cross-chain message is executed successfully on the destination chain to receive reimbursement for the order fill.

In particular, if the default bridge is Wormhole, the solver must pass a valid `signedQuote` as `bridgeAdapterArgs_`. If the `signedQuote` is invalid or if there is not enough `msg.value` provided, the message will not be automatically executed on the origin chain. In this case, the solver must manually execute the message on the origin chain to receive reimbursement. If the signedQuote is passed correctly, the message should be executed automatically. However, it is in the interest of the solver to verify the execution and execute it manually if necessary.

If the default bridge is Hyperlane, the message should be executed automatically as long as enough `msg.value` is provided. However, solvers should still verify that the message was executed successfully and trigger manual execution if necessary.

Solvers should be particularly vigilant in case the origin chain becomes congested or has a sudden gas price increase. This could increase the risk of the bridged message not being executed within the expected time.